2026 年“华数杯”大学生数学建模竞赛

所属类别	本科组
参赛编号	CM******

基于 B*-tree 表示与模拟退火的 VLSI 布图规划

摘　要

布图规划要在芯片轮廓内确定各功能模块的位置与方向，是电子设计自动化的核心环节。难点在于不重叠约束是析取形式，写成混合整数规划时约束规模是模块数的平方，上百个模块已无法在可接受时间内求解。本文把四个小问统一到一套框架下：用**B*-tree 布局表示**使任何一个解都自动不重叠，把析取约束从模型中消去；再用**模拟退火**在树结构与方向变量构成的离散空间上搜索，四个小问只更换代价函数或解码器。全部结果由一份独立程序重新校验，检查模块齐全、尺寸一致、两两不重叠、不越出轮廓，并从坐标重算线长。

针对问题一，题目要求在轮廓不固定时使包围矩形面积最小，面积相同时长宽比越接近 1 越好。首先建立**以面积为主、长宽比为辅的布图规划模型**，然后用 B*-tree 加轮廓打包把不重叠约束化去，最后用模拟退火配合按行排布的初始解求解。三组芯片的包围矩形依次为 419×439、403×453、539×537，面积 **183 941**、**182 559**、**289 443**，长宽比 0.954、0.890、1.004，对应死区比例 **2.47%**、**3.91%**、**5.96%**，其中两组低于参考文献报告的典型水平。

针对问题二，题目要求在死区比例 0.15 的固定正方形轮廓内使总半周长线长最小。建立**带越界惩罚的线长最小化模型**，用**两段退火**求解：前一段只把布局压进轮廓，后一段专压线长；退火后做三步后处理，依次是整体平移、滑动精修与拔出重插，把轮廓内剩余的空隙用起来。三组芯片的总线长依次为 **200 854**、**369 582**、**517 042**。

针对问题三，题目要求求出能容纳全部模块的最小死区比例及对应布局。本文先证明轮廓为正方形时**死区比例关于边长单调递增**，于是最小死区比例等价于能装下全部模块的最小正方形边长，不必对死区比例二分再逐个判定可行性；据此把**最小包围正方形边长的平方**直接作为退火目标，一次求解即可。三组芯片的最小死区比例依次为 **2.53%**、**5.24%**、**6.35%**，在该轮廓下重新优化线长的结果见表 4。

针对问题四，题目要求把模型推广到 L 型与 T 型模块并给出题给算例的最优布图。模型只需改三处：方向变量由二值扩展为四值，不重叠条件改写为**单元格集合不相交**，解码器换成**直角多边形的左下角填充**，搜索算法沿用问题一。题给四个模块的总面积 24 是包围矩形面积的下界，整数网格上的**分支定界**在 24 处找到可行摆放，故最小面积恰为 **24**，轮廓 4×6，死区为零，退火解与之一致，该解为全局最优。

为说明方法选择的依据，本文把模拟退火与遗传算法、禁忌搜索、多起点爬山、切片式波兰表达式退火、构造式贪心放在同一目标函数与同一求值次数预算下对照，每种方法各做 16 次独立重启。面积目标上退火在三组算例上都最好，切片表示因为只能描述一刀切的分割结构，死区比例始终高出十个百分点以上。最后对长宽比权重与重启次数做了灵敏度分析，并给出线长随轮廓收紧而上升的定量关系：死区由 0.15 收紧到最小值时，三组芯片的总线长分别上升 38.0%、16.2% 与 9.5%，说明面积与线长之间存在实质取舍。

关键词：　布图规划、B*-tree、模拟退火、半周长线长、死区比例

一、问题重述

1.1 问题背景

超大规模集成电路把大量电路单元集成在一块芯片上。设计复杂度上升以后，芯片设计已经离不开电子设计自动化工具，而布图规划是其中最靠前也最关键的一步：它决定各功能模块摆在哪里、朝向如何，后续的布局与布线都在此基础上展开。布图质量直接影响芯片面积与互连线长，面积决定成本，线长决定延迟与功耗。

布图规划的困难来自约束形式。任意两个模块不能重叠，这个条件是四个不等式的析取，写成混合整数规划需要为每一对模块引入额外的 0-1 变量，约束条数随模块数平方增长。上百个模块时精确求解已不现实，因此工程上普遍改用一种能自动满足不重叠条件的布局表示，再在表示空间上做启发式搜索。

1.2 问题要求

给定一组矩形硬模块，每个模块允许旋转 90 度，另有一组位置固定的端子只参与线长计算。模块之间的连接关系由线网描述，每条线网连接若干引脚，引脚位于所属模块的几何中心，线网的线长用其所有引脚外接矩形的半周长估计，记为 HPWL。四个小问依次要求：

1. 轮廓不固定且模块之间没有连接关系时，如何摆放模块使包围矩形面积最小，面积相同时长宽比越接近 1 越好；对 n100、n200、n300 三组芯片分别给出面积、长宽比与布图。
2. 轮廓固定为正方形，边长由总面积与死区比例 0.15 按公式确定，如何摆放模块使总 HPWL 最小，同时不重叠、不越出轮廓；给出三组芯片的总线长与布图。
3. 在问题二的基础上，求使可行布局仍然存在的最小死区比例，并在该轮廓下更新总 HPWL 与布图。
4. 模块可为 L 型与 T 型且允许 90、180、270 度旋转时，讨论问题一的模型如何修正，并对题给的四个模块求最小包围矩形面积。

二、问题分析

2.1 问题一的分析

轮廓自由时的目标是最小化包围矩形面积，长宽比只在面积相同时作为次级判据。难点不在目标函数，而在不重叠约束的形式。

首先，模块总面积 A 是包围矩形面积的下界，因此这一问等价于把死区比例压到最低。其次，直接对模块坐标做连续优化会被析取的不重叠约束卡住，需要换一种布局表示。本文采用 B*-tree：它是一棵有序二叉树，左孩子表示紧贴父模块右侧的模块，右孩子表示与父模块左边界对齐并位于其上方的模块，纵坐标由一条从左到右的轮廓线决定。这样任何一棵树解码出来都是不重叠的紧凑布局，不重叠约束被表示本身消化掉。最后，在树结构与方向变量构成的离散空间上用模拟退火搜索，扰动包括翻转模块方向、交换两个节点承载的模块、把一个节点摘下来重新挂到别处。

需要注意两点。一是解码必须按先序遍历且左子树优先，顺序压反虽然仍能得到合法布局，但质量明显变差。二是初始解不能用常见的完全二叉树：B*-tree 的宽度只由最长左链决定，而平衡树的左链长度是 $O(\log n)$，于是全部模块被迫竖直堆叠，解码出来是又窄又高的长条。本文改用按行排布的初始解，每行是一条左链，行首之间用右孩子相连，初始布局就是二维的。

2.2 问题二的分析

轮廓固定后，可行解不难找到，难点转为把线长压下去。

首先，轮廓边长由公式直接给出，死区比例 0.15 是相当宽松的设定，找到可行布局并不困难，难点在于把线长压下去。其次，端子位置固定，使得 HPWL 不再具有平移不变性，布局整体挪动会改变线长，这一点可以利用。最后，B*-tree 解码出的布局总是紧贴左下角，轮廓内剩余的空隙没有被利用，因此在退火之后还需要后处理。

本文的做法是：把越界量写成惩罚项并入代价函数；退火分两段，前一段只压最小包围正方形边长把布局塞进轮廓，后一段改压 HPWL；退火结束后先做整体平移，再做滑动精修，逐个模块在不与他人冲突的空隙内沿两个坐标方向分别移到线长最优位置。

2.3 问题三的分析

这一问看似要对死区比例做二分再反复判定可行性，实际上有一个单调性可以直接化简。

关键的观察是：轮廓为正方形时边长 $S=\sqrt{A(1+\Gamma)}$，于是 $\Gamma=S^2/A-1$ 关于 S 严格单调递增。因此最小可行死区比例等价于能装下全部模块的最小正方形边长，不需要对 Γ 做二分再反复判定可行性，直接把 $\max(W,H)^2/A$ 当作退火目标优化一次即可，其取值本身就是 $1+\Gamma$。求得 $\Gamma_{\min}$ 后，把它代回问题二的模型重跑一次线长优化。

由于退火是启发式算法，得到的是 $\Gamma_{\min}$ 的一个上界，即在给定计算预算下搜到的最紧可行轮廓。这一化简的依据与效果见图 1。

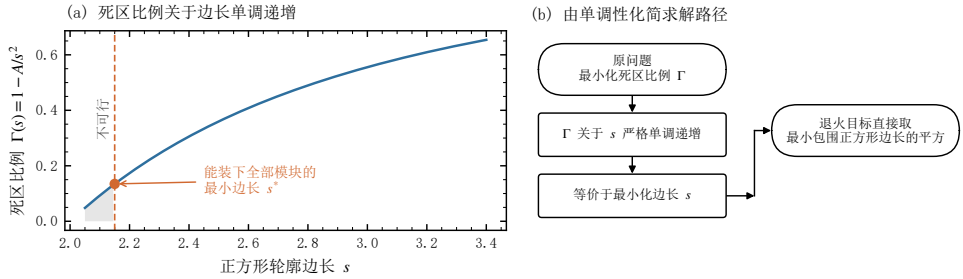


图 1　死区比例关于轮廓边长的单调性及由此化简的求解路径

2.4 问题四的分析

模块由矩形变为直角多边形后，模型需要修改三处，搜索算法不必重新设计。

第一处是方向变量，由二值扩展为四值，几何用单元格集合描述，旋转 90 度对应坐标映射 $(x,y)\mapsto(-y,x)$ 后平移回第一象限。第二处是不重叠条件，改写为两个平移后的单元格集合不相交；若把直角多边形拆成矩形的并，条件就退化为矩形之间两两不重叠，与原模型同形，只是约束条数变多。第三处是解码器，B*-tree 的轮廓打包假设模块是矩形，遇到凹形模块会在凹口浪费面积，因此换成直角多边形的左下角填充，即按给定顺序逐个放置，每个模块取最靠下、同高时最靠左的可行位置。

题给四个模块的边长都是小整数，可以在整数网格上做精确的分支定界，得到严格最优解并与退火解互相印证。方向变量的扩展与不重叠条件的改写见图 2。

2.5 问题总体思路图

四问共用同一套表示与搜索框架，只更换代价函数或解码器，整体路径见图 3。

三、模型假设

1. 所有功能模块都是硬模块，尺寸给定且不可缩放，只允许旋转，不允许镜像。

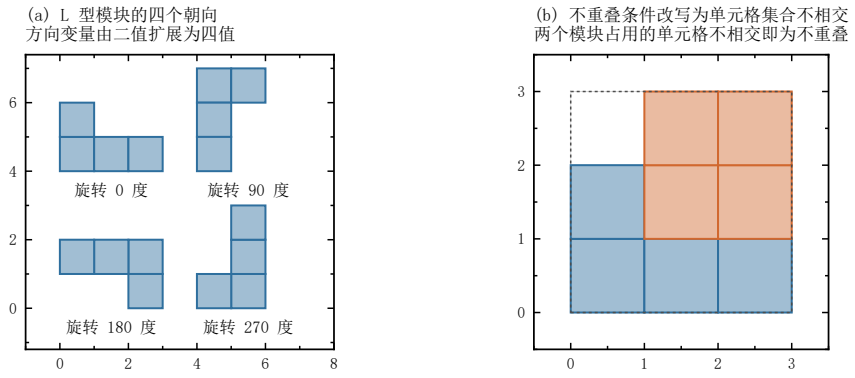


图 2　直角多边形模块的四个朝向与单元格集合不相交条件

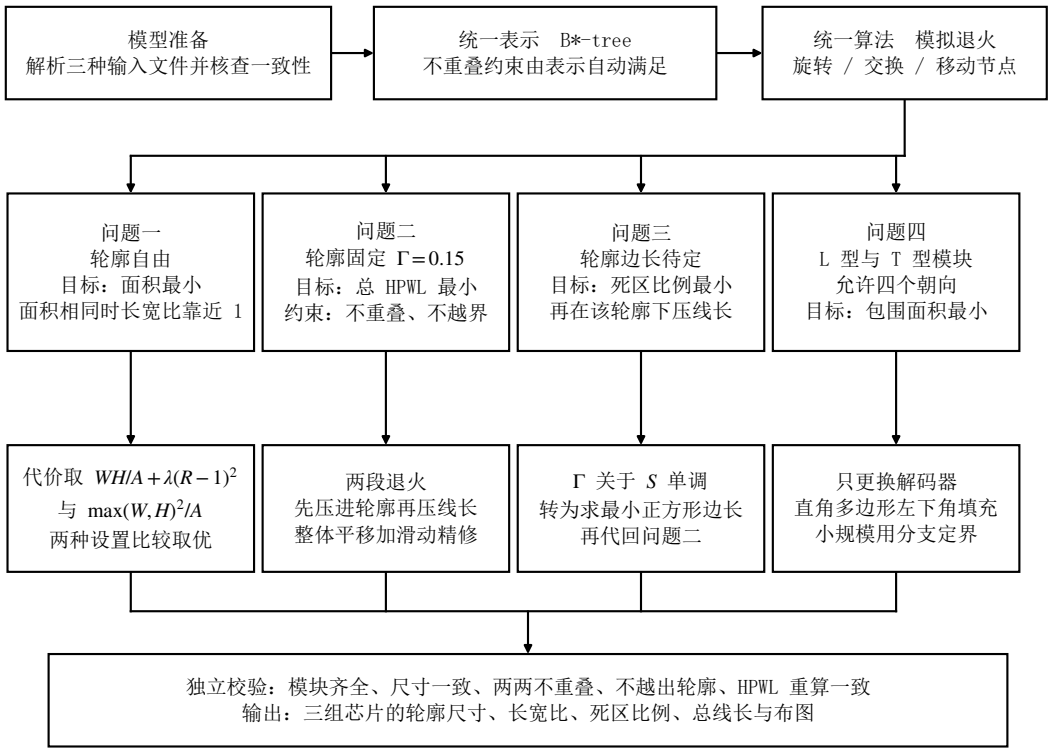


图 3　文章求解思路框架图

2. 每个模块的引脚集中在其几何中心，端子的引脚位于附件给出的坐标点上。
3. 线网的线长用引脚外接矩形的半周长估计，不考虑实际布线的绕行与层间过孔。
4. 端子只参与线长计算，不占用芯片面积，也不参与不重叠约束。
5. 芯片轮廓的左下角固定在坐标原点，模块坐标取非负值。
6. 不考虑布线拥塞、时序、功耗与热分布等后续工序的约束。

四、符号说明

符号	说明	单位
n	硬模块个数	个
$w_i,\ h_i$	模块 i 的原始宽与高	长度单位
r_i	模块 i 的方向变量	无量纲
$\tilde{w}_i,\ \tilde{h}_i$	模块 i 摆放后的实际宽与高	长度单位
$x_i,\ y_i$	模块 i 左下角坐标	长度单位
$W,\ H$	布局包围矩形的宽与高	长度单位
A	模块总面积 $\sum_i w_i h_i$	面积单位
Γ	死区比例	无量纲
S	正方形轮廓的边长	长度单位
R	轮廓长宽比 $\max(W,H)/\min(W,H)$	无量纲
N_k	第 k 条线网的引脚集合	—
T	固定端子集合	—
HPWL	全部线网的半周长线长之和	长度单位

五、模型的准备

四个小问共用同一套基础量与同一种布局表示，本节先统一交代。

5.1 决策变量与几何量

每个模块有三个决策量：左下角坐标 (x_i, y_i) 与方向变量 r_i。矩形模块只允许 90 度旋转，故 $r_i \in \{0, 1\}$，实际宽高为

$$\tilde{w}_i = (1 - r_i)w_i + r_i h_i, \qquad \tilde{h}_i = (1 - r_i)h_i + r_i w_i. \tag{1}$$

包围矩形的宽高由全部模块决定：

$$W = \max_i (x_i + \tilde{w}_i), \qquad H = \max_i (y_i + \tilde{h}_i). \tag{2}$$

模块总面积 $A = \sum_i w_i h_i$ 是常数，它给出轮廓面积的下界。死区比例定义为轮廓面积超出该下界的比例，正方形轮廓的边长与死区比例的关系为

$$S = \sqrt{A\,(1 + \Gamma)}, \qquad \text{等价地} \qquad \Gamma = \frac{S^2}{A} - 1. \tag{3}$$

5.2 不重叠约束及其困难

任意两个模块 $i \neq j$ 必须满足

$$x_i + \tilde{w}_i \le x_j \quad \vee \quad x_j + \tilde{w}_j \le x_i \quad \vee \quad y_i + \tilde{h}_i \le y_j \quad \vee \quad y_j + \tilde{h}_j \le y_i. \tag{4}$$

式(4)是四个线性不等式的析取。若用大 M 方法线性化，每一对模块需要引入 4 个 0-1 变量和 4 条约束，总规模是 $O(n^2)$。本题最大算例有 300 个模块，对应约 1.8×10^5 个附加 0-1 变量，混合整数规划无法在可接受时间内求得可行解，因此必须换一种解的表示。

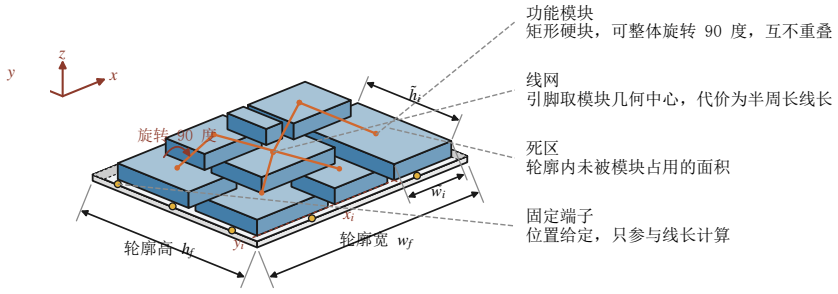


图 4　布图规划的立体场景。模块摆在芯片轮廓内互不重叠，未被占用的部分是死区，线网连接模块中心与固定端子

5.3　半周长线长

布图规划的实物形态见图 4：模块像一块块硬砖摆在芯片上，既不能压边也不能叠放，摆得越紧面积越小，摆得越靠近相连的模块线长越短。

本题涉及的几何对象与两个目标量集中画在图 5 中：芯片轮廓由 w_f 与 h_f 界定，轮廓内未被模块占用的部分是死区，固定端子分布在轮廓四边附近且只参与线长计算；一条线网的半周长就是其全部引脚外接矩形在两个方向上的跨度之和。

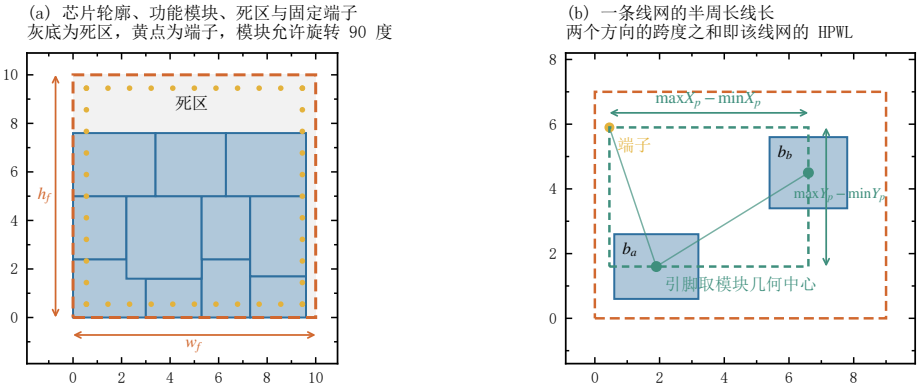


图 5　布图规划的构成与半周长线长的定义

第 k 条线网连接一组引脚。模块 i 的引脚坐标为 $\left(x_i + \tilde{w}_i/2,\ y_i + \tilde{h}_i/2\right)$，端子的引脚坐标由附件直接给出。该线网的线长与总线长为

$$\mathrm{HPWL}_k = \left(\max_{p\in N_k} X_p - \min_{p\in N_k} X_p\right) + \left(\max_{p\in N_k} Y_p - \min_{p\in N_k} Y_p\right), \qquad \mathrm{HPWL} = \sum_k \mathrm{HPWL}_k. \tag{5}$$

附件中线网以两点网为主，最大度数只有 6，因此式(5)可以逐网独立计算，一次全量求值的代价与引脚总数同阶，即使每次退火迭代都重算也不昂贵。

5.4　B*-tree 表示与解码

B*-tree 是一棵有序二叉树，节点与模块一一对应，规则为：

- 根节点对应左下角的模块，坐标取 $(0, 0)$；
- 若 j 是 i 的左孩子，则 $x_j = x_i + \tilde{w}_i$，即 j 紧贴 i 的右侧；
- 若 j 是 i 的右孩子，则 $x_j = x_i$，即 j 与 i 左边界对齐并位于其上方。

纵坐标由轮廓线决定。维护一条从左到右的分段常值函数 $c(\cdot)$ 表示每个横坐标上已被占用的最大高度，放置模块 i 时取

$$y_i = \max_{x \in [x_i,\ x_i+\tilde{w}_i)} c(x), \tag{6}$$

随后把该区间的轮廓抬高到 $y_i + \tilde{h}_i$。这三步的几何含义见图 6：横坐标由父模块给定，纵坐标由轮廓线在该区间上的最大值定出，模块落位后轮廓被抬高，因此后放的模块不可能压到先放的模块上。

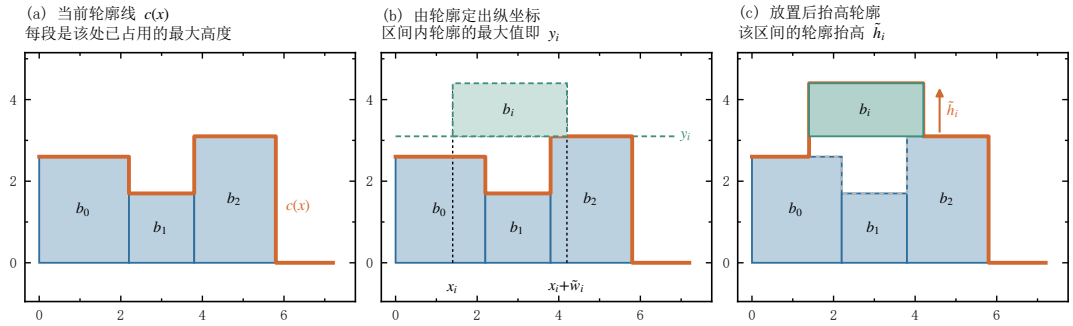


图 6　轮廓打包的三步：定横坐标、由轮廓定纵坐标、抬高轮廓

这样得到两条性质。其一，任何一棵 B*-tree 加一组方向变量，解码后一定不重叠，因为每个模块都放在当前轮廓之上而轮廓只升不降；于是式(4)被表示本身消化，不再需要写成约束。其二，解码按先序遍历且左子树优先，每个模块只处理一次，轮廓用有序数组维护，一次解码的时间为 $O(n \log n)$。一棵树与它解码出的布局的对应关系见图 7。

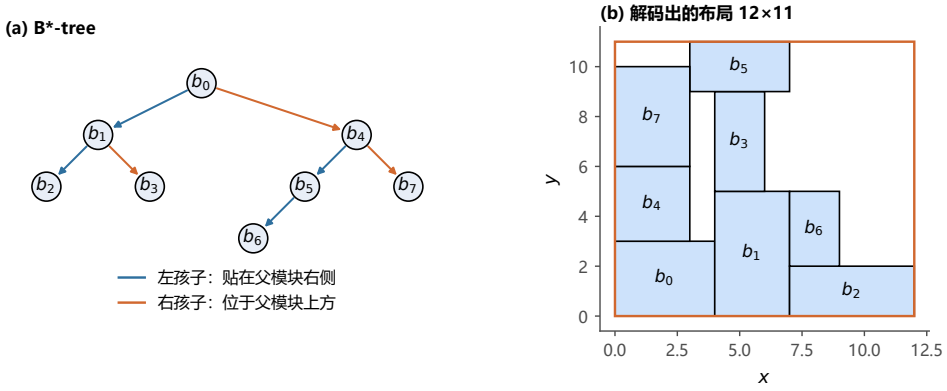


图 7　B*-tree 表示与解码出的布局

于是四个小问的求解都归结为：在树结构与方向变量构成的离散空间上，用模拟退火最小化各自的代价函数。三种邻域操作为翻转某个模块的方向、交换两个节点承载的模块、摘下一个节点重新挂到另一节点的左孩子或右孩子位置，示意见图 8。

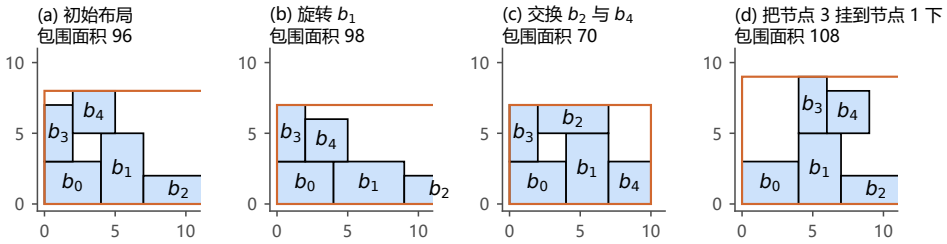


图 8　三种邻域操作及其对布局的影响

四个小问共用的求解流程见图 9。

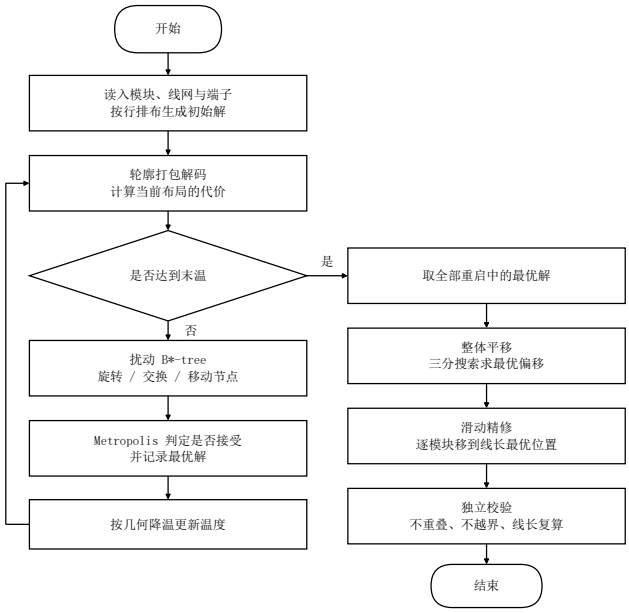


图 9　模拟退火求解流程

六、模型的建立与求解

6.1　问题一模型的建立与求解

6.1.1　问题一思路框架

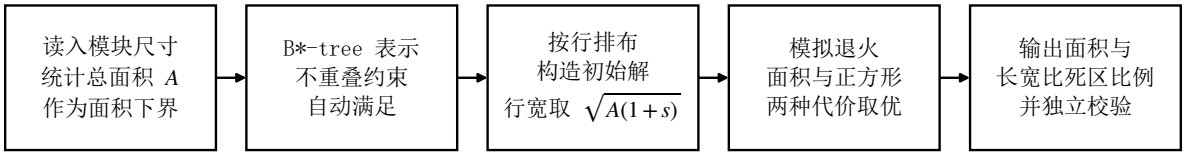


图 10　问题一思路框架图

6.1.2　数据分析

附件三组芯片的规模如表 1 所示。头部声明的模块数、端子数、线网数、引脚数与实际行数逐项一致，每条线网声明的度数与实际引脚数一致，‘.pl‘ 文件恰好覆盖全部端子，线网中没有出现未定义节点，全部硬模块都是四角点矩形。这说明前三问只需按矩形处理。

表 1　三组芯片的规模

项目	n100	n200	n300
硬模块数	100	200	300
固定端子数	334	564	569
线网数	885	1 585	1 893
引脚总数	1 873	3 599	4 358
模块总面积 A	179 501	175 696	273 170
$\sqrt{A}$	423.68	419.16	522.66

模块尺寸均为整数，长宽比最大不到 4，没有细长条；最大模块的短边远小于轮廓边长，因此单个模块不会成为装配瓶颈，面积利用率主要由摆放策略决定。

6.1.3 最小面积布图模型的建立

问题一的模型整理如下：

$$
\begin{cases}
\min \quad W \cdot H, \qquad \text{面积相同时取} R=\dfrac{\max(W,H)}{\min(W,H)} \text{ 最小} \\
\text{s.t.} \quad \begin{cases}
x_i+\tilde{w}_i \le x_j \ \vee \ x_j+\tilde{w}_j \le x_i \ \vee \ y_i+\tilde{h}_i \le y_j \ \vee \ y_j+\tilde{h}_j \le y_i, & \forall\, i<j \\
W=\max\limits_i (x_i+\tilde{w}_i), \quad H=\max\limits_i (y_i+\tilde{h}_i), & \\
\tilde{w}_i=(1-r_i)w_i+r_i h_i, \quad \tilde{h}_i=(1-r_i)h_i+r_i w_i, & \forall\, i \\
x_i \ge 0,\ y_i \ge 0,\ r_i \in \{0,1\}. &
\end{cases}
\end{cases}
\tag{7}
$$

用 B*-tree 表示后，析取约束自动满足，决策变量变为树结构 $\mathcal{T}$ 与方向向量 $\boldsymbol{r}$，目标写成可退火的标量代价

$$
\mathrm{cost}_1(\mathcal{T},\boldsymbol{r})=\frac{WH}{A}+\lambda\,(R-1)^2 . \tag{8}
$$

平方项使得长宽比接近 1 时几乎不起作用，明显偏离 1 时迅速增大。

本文还使用了第二个代价函数，即问题三所用的最小包围正方形

$$
\mathrm{cost}_1'(\mathcal{T},\boldsymbol{r})=\frac{\max(W,H)^2}{A}. \tag{9}
$$

式(8)把面积与长宽比放在一起，两项可以互相交换，单次退火的方差较大；式(9)只有一个量要压，收敛更稳定。最终答案取两种代价函数在全部重启中的最好解，判优规则按题目要求先比面积再比长宽比。

6.1.4 模型的求解

求解算法为模拟退火。初始解按行排布构造，行宽取 $\sqrt{A(1+s)}$，s 在每次重启时于 $[0.02, 0.30]$ 内随机取值。温度按几何降温，初温由随机单步扰动的平均代价变化定出，末温取初温的 10^{-6}；降温结束后再做一段只接受不变差的贪心爬坡。每个算例做 20 次独立重启并取优，全部计算在 20 线程上并行。

三组芯片的结果见表 2 与图 11。

表 2　问题一结果：三组芯片的轮廓与死区比例

算例	轮廓 $W\times H$	轮廓面积	长宽比 W/H	死区比例	文献典型值
n100	419×439	183 941	0.954	2.47%	4.9%–5.1%
n200	403×453	182 559	0.890	3.91%	4.9%–5.1%
n300	539×537	289 443	1.004	5.96%	5.2%–5.3%

死区并不集中在某一处，而是以细缝的形式散布在模块之间，见图 12。这说明剩余的改进空间来自模块之间的配合，而不是某一块大的空白，也解释了为什么死区比例继续下降会越来越困难。

6.1.5 模型的检验

检验分三个层次。

第一，几何合法性。求解器输出的每个布局都用一份独立的 Python 程序重新读入原始文件并检查：输出模块集合与附件完全一致、每个模块的宽高等于原始宽高或其交换且与方向变量吻合、任意两个模块不重叠。三组结果全部通过。

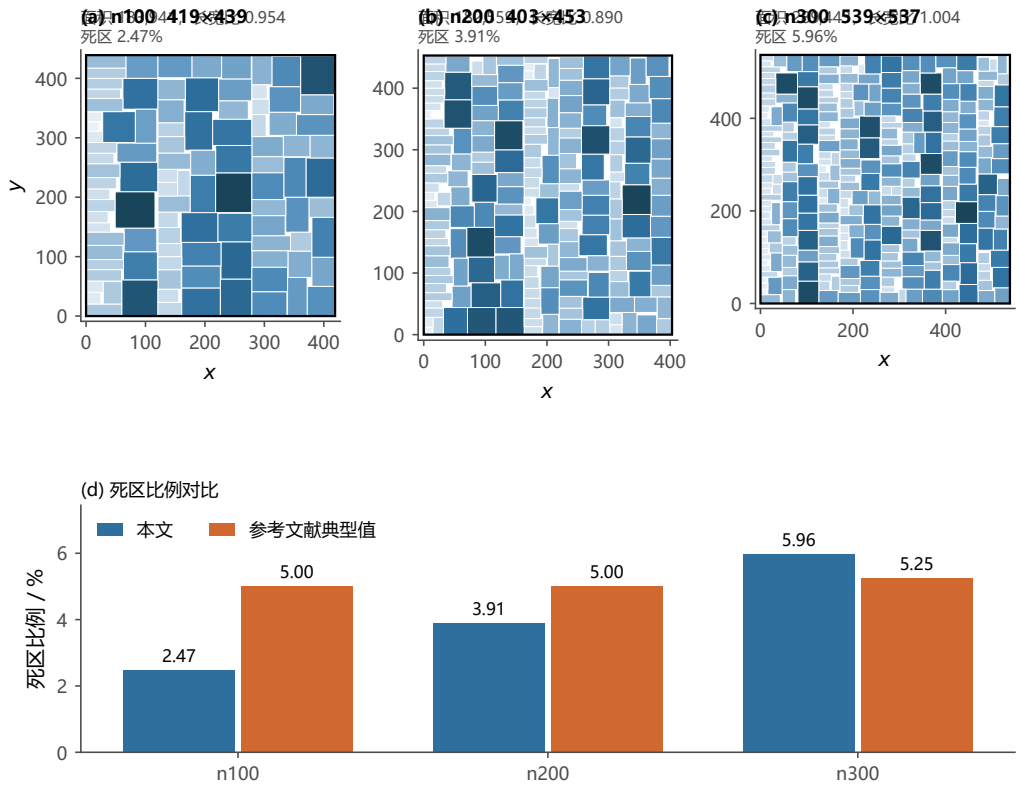


图 11　问题一结果图：三组芯片的布图与死区比例对比

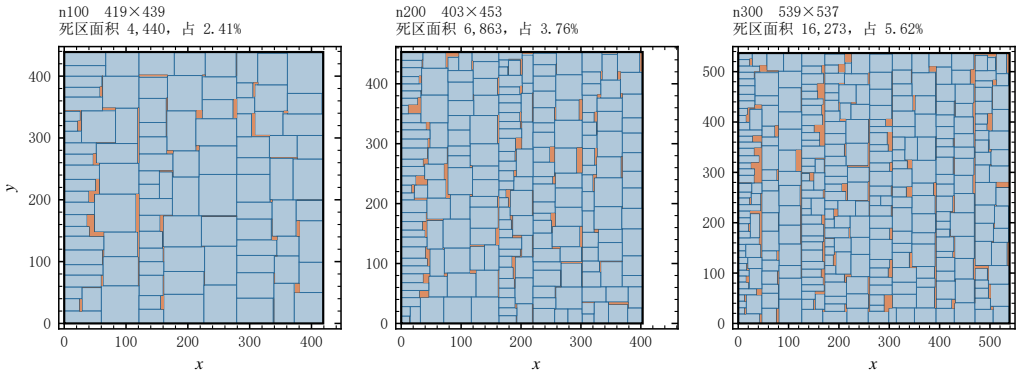


图 12　问题一结果的死区分布，橙色为未被模块占用的部分

第二，实现正确性。解码顺序与初始解构造是两处容易出错的地方，本文用对照实验确认其影响。同一棵 B*-tree 按左子树优先解码得到 442×644，死区 58.6%；按右子树优先解码得到 442×996，死区 145.3%，两者都不重叠但质量相差悬殊，说明顺序必须取前者。同样地，完全二叉树初始解解码出 283×2020、长宽比 7.1 的长条，按行排布初始解解码出 442×543、长宽比 1.2 的方块，见图 13 与图 14。

第三，结果合理性。文献 [2] 在同一组基准上报告的无固定轮廓死区比例为 4.9% 到 5.3%，本文三组结果为 2.47%、3.91%、5.96%，处于同一量级，其中两组更优。

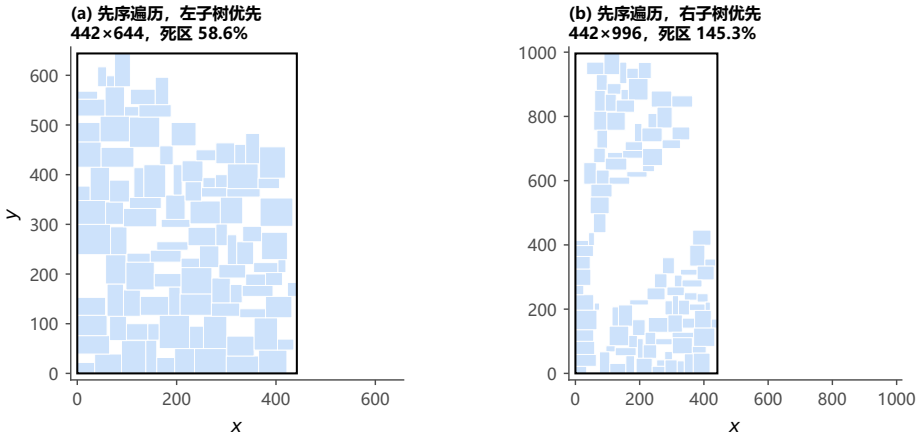


图 13　解码遍历顺序对布局质量的影响

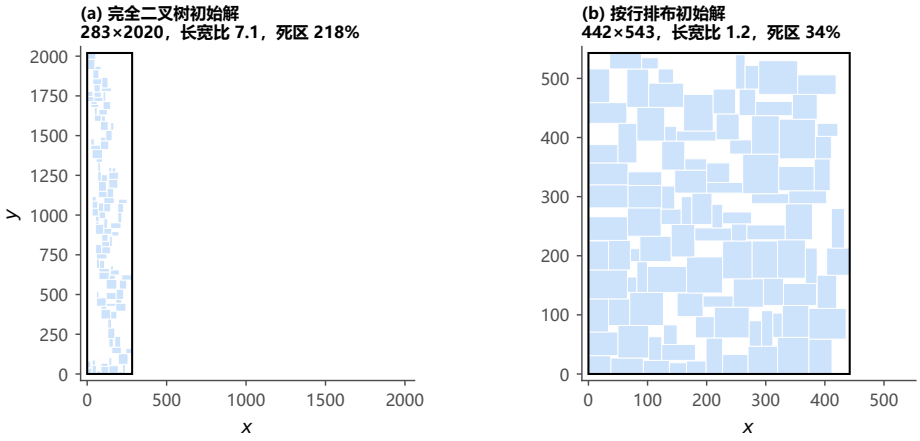


图 14　初始解构造方式对布局质量的影响

6.2　问题二模型的建立与求解

6.2.1　问题二思路框架



图 15　问题二思路框架图

6.2.2　固定轮廓下的线长最小化模型的建立

轮廓边长由式(3)取 $\Gamma = 0.15$ 得到。模型为

$$
\begin{cases}
\min \quad \mathrm{HPWL} = \displaystyle\sum_{k} \left(\max_{p\in N_k} X_p - \min_{p\in N_k} X_p + \max_{p\in N_k} Y_p - \min_{p\in N_k} Y_p \right) \\
\text{s.t.} \begin{cases}
\text{式(4)的不重叠约束}, & \forall\, i<j \\
0 \le x_i, \quad x_i + \tilde{w}_i \le S, \quad 0 \le y_i, \quad y_i + \tilde{h}_i \le S, & \forall\, i \\
S = \sqrt{A \times 1.15}, \qquad r_i \in \{0,1\}.
\end{cases}
\end{cases}
\tag{10}
$$

用 B*-tree 表示后不重叠自动满足，越界只需比较解码得到的 W, H 与 S。把越界

量写成相对长度

$$\delta_x = \frac{\max(0,\, W - S)}{S}, \qquad \delta_y = \frac{\max(0,\, H - S)}{S}, \tag{11}$$

代价函数取

$$\mathrm{cost}_2 = \frac{\mathrm{HPWL}}{\mathrm{HPWL}_{\mathrm{norm}}} + 60\left(\delta_x^2 + \delta_y^2\right) + 6\left(\delta_x + \delta_y\right), \tag{12}$$

其中 $\mathrm{HPWL}_{\mathrm{norm}}$ 取初始阶段随机扰动得到的平均线长。只有 $\delta_x = \delta_y = 0$ 的解才被记录为可行最优解。

6.2.3 模型的求解

退火分两段。第一段占 30% 的降温步，只优化 $\max(W,H)^2/A$，目的是先把布局压进轮廓；若此时仍未进入轮廓则自动延长该段。第二段切换到 cost_2，并按新目标重新估一次初温。

退火结束后做两步后处理，两步都严格保持不重叠与不越界。

第一步是整体平移。布局占 $W \times H$，轮廓为 $S \times S$，尚余 $S - W$ 与 $S - H$ 的空间。端子位置固定，所以 HPWL 不是平移不变的。对每条线网，其 x 方向跨度关于平移量 Δx 是分段线性凸函数，求和后仍然凸，因此用三分搜索即可求得最优平移。

第二步是滑动精修。逐个模块计算它在不与其他模块冲突、不越出轮廓的前提下可移动的区间，再在该区间内把它移到线长最优位置；单方向的目标同样是分段线性凸函数，最优点必在断点上，枚举断点即可。反复扫描直到没有模块可动。实测这一步在三组算例上分别再降低约 1% 的总线长。

第三步是拔出重插。滑动精修只让模块在原位附近平移，跨不过其他模块。这一步把模块整个取走，在剩下模块形成的所有角点位置里挑一个能装下、又能使总线长下降最多的位置放回去，两个朝向都试，只接受严格下降的移动。候选位置的横坐标取自 $\{0\}$ 与其余模块的右边界，纵坐标取自 $\{0\}$ 与其余模块的上边界，因为最优摆放总可以推到某个角点上。把模块中心记为 (p, q)，它所连线网去掉自己以后的包围盒记为 $[l_k, r_k] \times [b_k, t_k]$，则该模块贡献的可变线长为

$$g(p,q) = \sum_k \Big[\max(p - r_k, 0) + \max(l_k - p, 0) + \max(q - t_k, 0) + \max(b_k - q, 0)\Big], \tag{13}$$

关于 p 与 q 可分且分段线性凸，所有候选位置的 g 可以一次算完，再按 g 从小到大做重叠检查，取第一个合法的即可，不必逐个试。

这一步的几何含义与实测效果见图 16。滑动精修只能让模块在原位附近平移，遇到相邻模块就停住；拔出重插允许模块跨过其他模块落到远处的空位上，n300 有 13 个模块因此换了位置，总线长再降 0.56%。

两步后处理的几何含义见图 17。B*-tree 解码出的布局总是紧贴左下角，轮廓右侧与上方留有 $S - W$ 与 $S - H$ 的空白；端子固定不动使线长不再平移不变，于是整体挪动布局就能改变线长。滑动精修则利用模块之间的空隙，把单个模块移到使其所连线网跨度最小的位置。

表 3　问题二结果：固定轮廓下的总线长

算例	轮廓边长 S	轮廓面积	布局 $W \times H$	总 HPWL
n100	454.34	206 426	452×453	200 854.5
n200	449.50	202 050	449×448	369 582.0
n300	560.49	314 145	550×558	517 042.0

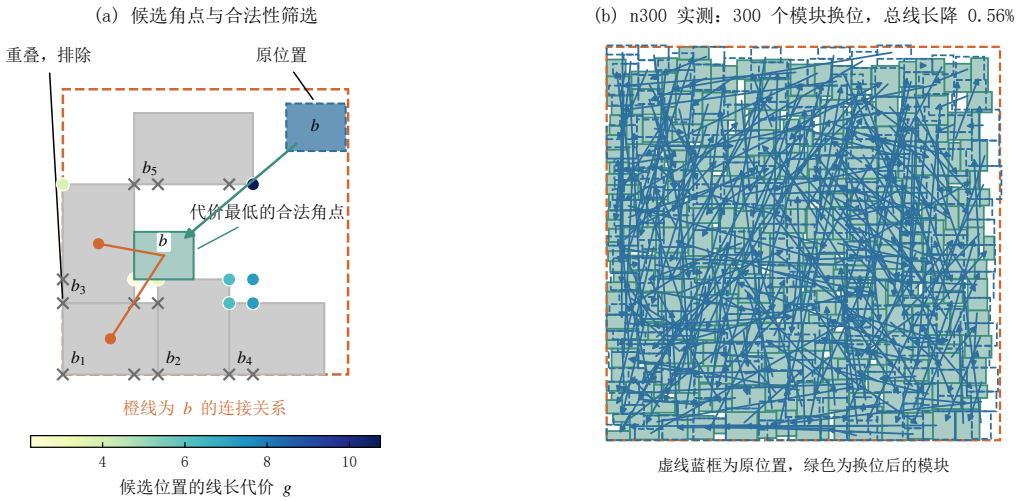


图 16　拔出重插精修：候选角点的筛选与 n300 上的实测位移

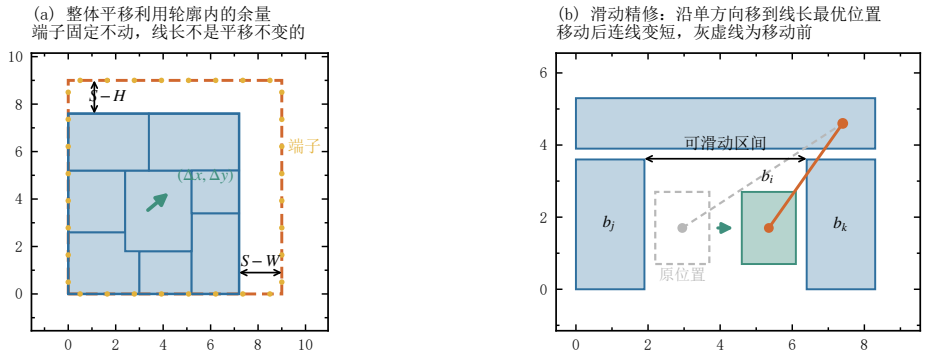


图 17　固定轮廓下的两步后处理：整体平移与滑动精修

三组芯片在固定轮廓内的布图见图 18，红色虚线为轮廓，橙色点为只参与线长计算的固定端子。

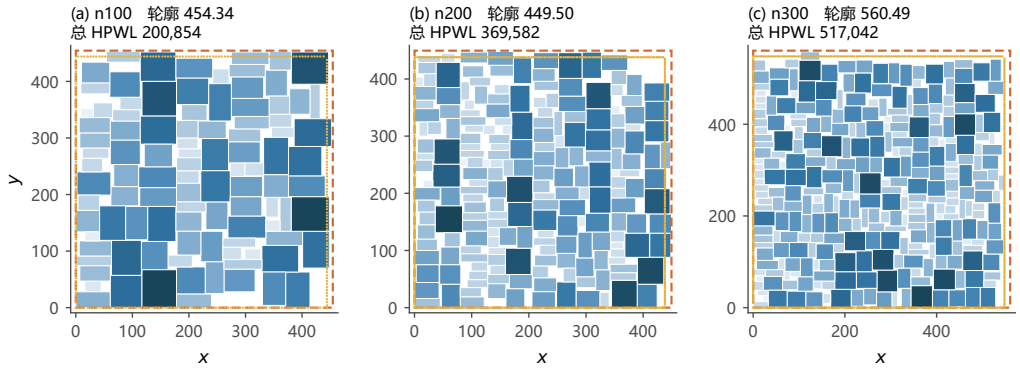


图 18　问题二结果图：固定轮廓内的布图，红色虚线为轮廓，橙色点为固定端子

把每个模块按它所连线网的总长着色，再画出每个模块指向其连接重心的位移，可以看出优化的效果，见图 19。线长大的模块集中在轮廓边缘，因为它们连的端子固定在四周；绝大多数模块的位移箭头很短，说明模块已经落在其连接对象的重心附近，继续挪动的收益有限。

6.2.4　模型的检验

除问题一的几何检验外，本问增加两项。

其一，越界检查。校验程序对每个模块检查 $x_i + \tilde{w}_i \le S$ 与 $y_i + \tilde{h}_i \le S$，三组结果

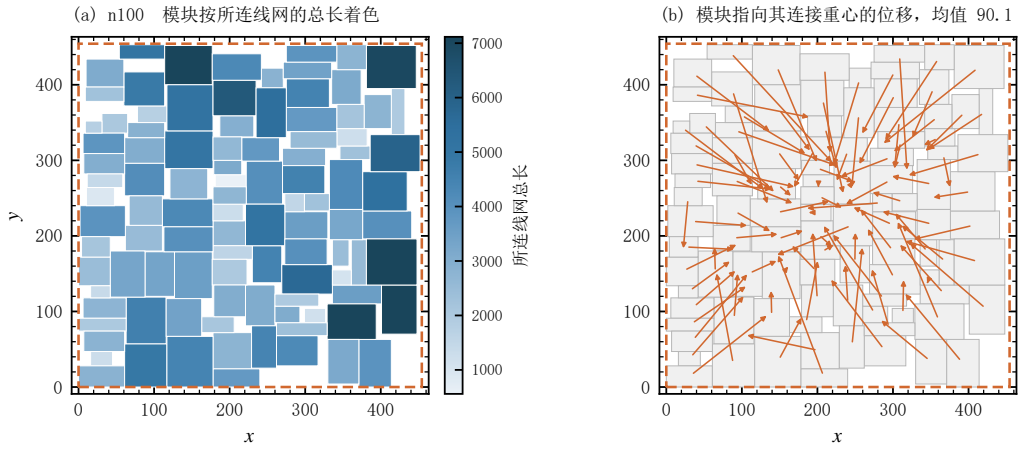


图 19　问题二结果的线长分布与模块到连接重心的位移，n100

全部通过。

其二，线长复算与量级核对。校验程序从坐标重新计算式(5)，与求解器报告值完全一致。文献 [2] 在同样的固定轮廓设定下给出了三组芯片的线长，但其长度单位与基准文件的原始单位之间存在未说明的换算系数，数值不能直接对齐，只能比较三组之间的相对量级。以 n100 为基准，文献的相对量级为 1 : 1.82 : 2.21，本文为 1.00 : 1.84 : 2.57，n100 与 n200 吻合良好，n300 偏高约 16%，说明该算例的线长仍有下降空间。

图 20 进一步给出单条线网的线长分布：与面积最优布局相比，线长最优布局把大量线网从中长线段压到短线段，分位数曲线整体下移。

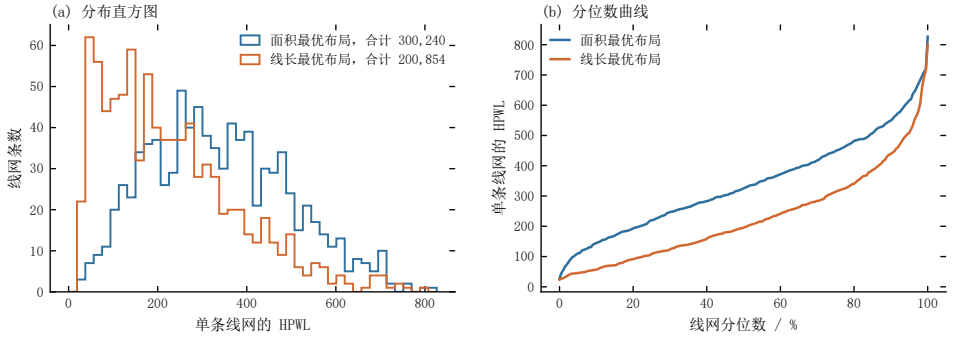


图 20　单条线网的线长分布与分位数曲线，n100

把面积最优布局与线长最优布局并排画出，并在两者上画同一组线网，可以直观看到两个目标的差别，见图 21。面积最优的布局把模块压得更紧，但没有考虑连接关系，连线明显更长；线长最优的布局利用轮廓内的余量把相连的模块拉近，代价是留下了更多空隙。

6.3　问题三模型的建立与求解

6.3.1　问题三思路框架

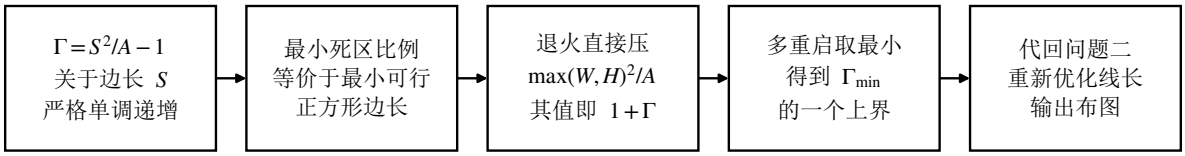


图 22　问题三思路框架图

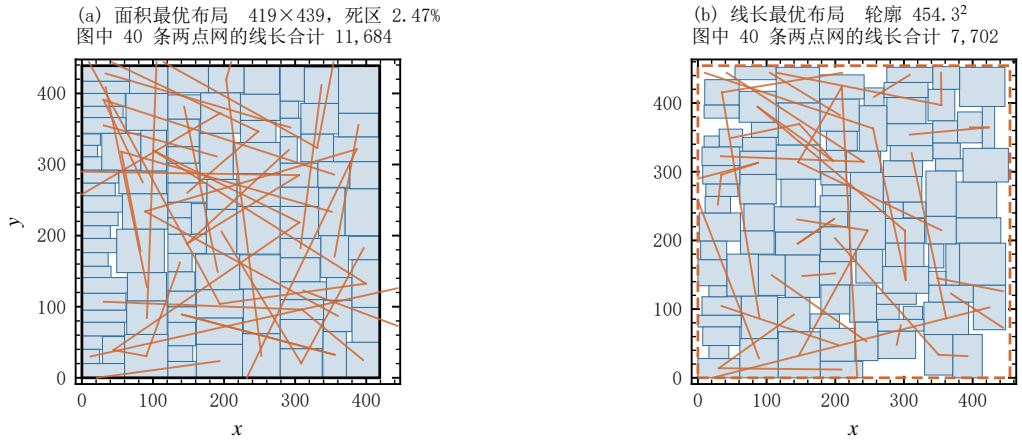


图 21　面积最优布局与线长最优布局的对比，图中画出随机抽取的 40 条两点网

6.3.2　最小死区比例模型的建立

这一问可以由一个单调性直接化简，不必对死区比例二分再逐个判定可行性。下面先给出这一等价转化。

命题. 设轮廓为边长 S 的正方形，模块总面积为 A，则死区比例 $\Gamma = S^2/A - 1$ 关于 S 严格单调递增；因此存在可行布局的最小死区比例

$$\Gamma_{\min} = \frac{S_{\min}^2}{A} - 1, \qquad S_{\min} = \min_{\mathcal{T},\boldsymbol{r}} \ \max\big(W(\mathcal{T},\boldsymbol{r}),\, H(\mathcal{T},\boldsymbol{r})\big). \tag{14}$$

证明. Γ 是 S 的严格增函数，所以 Γ 可行当且仅当 $S = \sqrt{A(1+\Gamma)}$ 可行。而边长为 S 的正方形能装下全部模块，当且仅当存在一个不重叠布局使 $W \le S$ 且 $H \le S$，即 $\max(W,H) \le S$。故可行边长的集合是 $[S_{\min}, +\infty)$，代入 $\Gamma = S^2/A - 1$ 即得式(14)。□

这条命题的意义在于：不必对 Γ 做二分再逐个判定可行性，只要把 $\max(W,H)$ 当作目标优化一次即可，省掉整整一层外循环。于是模型为

$$\begin{cases} \min \quad \dfrac{\max(W,H)^2}{A} \quad \big(=1+\Gamma\big) \\ \text{s.t.} \quad \begin{cases} \text{式(4)的不重叠约束}, & \forall\, i<j \\ W = \max\limits_i (x_i + \tilde{w}_i), \quad H = \max\limits_i (y_i + \tilde{h}_i), \\ x_i \ge 0,\ y_i \ge 0,\ r_i \in \{0,1\}. \end{cases} \end{cases} \tag{15}$$

求得 $\Gamma_{\min}$ 后，把它代回问题二的模型重新优化线长。

6.3.3　模型的求解

退火框架与问题一相同，只把代价函数换成 $\max(W,H)^2/A$。由于该目标只有一个量要压，收敛比问题一的加权代价更稳定，图 23 的收敛曲线对比说明了这一点。

求得 $\Gamma_{\min}$ 后把它代回问题二的模型重跑线长优化，轮廓按硬约束处理，即任何越出轮廓的解一律不接受。这里有一个绕不开的困难：从随机初始解出发，退火几乎不可能走进 $\Gamma_{\min}$ 对应的轮廓，因为该轮廓已经被模块填满，可行解在解空间里稀疏到几十次重启也碰不到一个。若改用软惩罚放宽，解又会停在比 $\Gamma_{\min}$ 松得多的地方，失去意义。

本文的做法是让求解器把最优解对应的 B*-tree 直接写成文件，再由下一次运行读回作为初始解。求 $\Gamma_{\min}$ 的那一次运行导出树，压线长的运行以它为起点，于是初始解

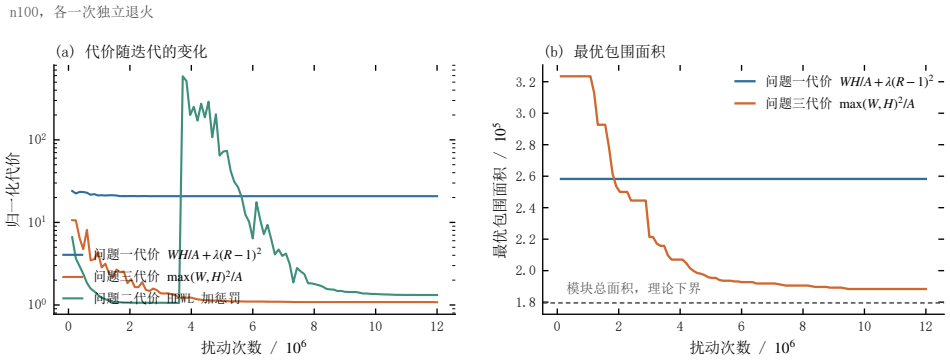


图 23　三种代价函数的退火收敛曲线，n100，各一次独立运行

本身就在轮廓内，退火全程只需在可行域内部搜索。这一步把线长压下去一到两成，见表 4，说明先前认为最小轮廓下布局已经没有优化余地的判断是错的：难的不是可行解少，而是找不到进入可行域的入口。

实现上有两处容易踩空。其一，轮廓边长要直接以整数给出，不能由死区比例反算。$\sqrt{A(1+\Gamma)}$ 在浮点下常落在比整数略小的位置，例如算出 430.99999，于是边长 431 的合法布局被判成越界，硬约束模式会报告找不到可行解。其二，初始解必须先记入历史最优。否则种子解虽然可行，一旦退火在高温段漂出轮廓且再未回来，最后返回的就是一个不可行解。这两点都曾使本文的求解失败，修正后才得到表中结果。

结果见表 4 与图 25。三组芯片把死区从 0.15 收紧到 $\Gamma_{\min}$，总线长分别上升 38.0%、16.2% 与 9.5%，说明面积与线长之间存在实质的取舍，三问最优结果的关系见图 24。需要说明的是，退火是启发式算法，表中的最小死区比例是在给定计算预算下搜到的最紧可行轮廓，是 $\Gamma_{\min}$ 的一个上界。

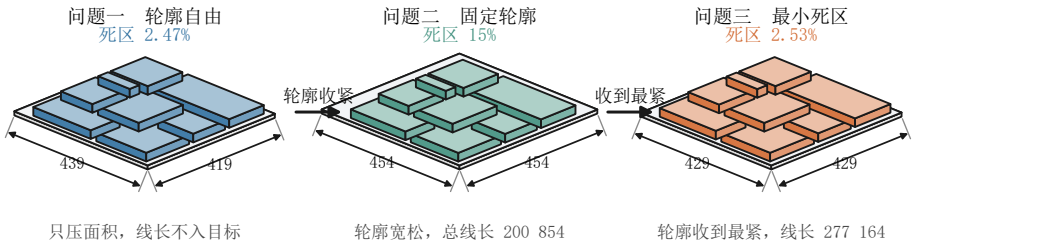


图 24　三问最优结果的关系。轮廓由自由到宽松再到最紧，面积随之下降，线长随之上升

表 4　问题三结果：最小死区比例与该轮廓下的总线长

算例	最小正方形边长	最小死区比例	该轮廓下总 HPWL	相对 $\Gamma=0.15$
n100	429	2.53%	277 164.5	+37.99%
n200	430	5.24%	429 584.5	+16.24%
n300	539	6.35%	566 130.5	+9.49%

6.3.4　模型的检验

其一，可行性检验。校验程序确认最小轮廓下的布局确实不重叠且不越界，即所报告的 $\Gamma_{\min}$ 确有对应的可行方案。

其二，单调性一致。三组芯片在 $\Gamma_{\min}$ 下的总线长均高于 $\Gamma=0.15$ 时的总线长，与轮廓收紧使模块可挪动余地变小的直觉一致，见表 4 最后一列。

其三，与问题一的交叉核对。问题一得到的包围矩形面积不小于 A，问题三得到的

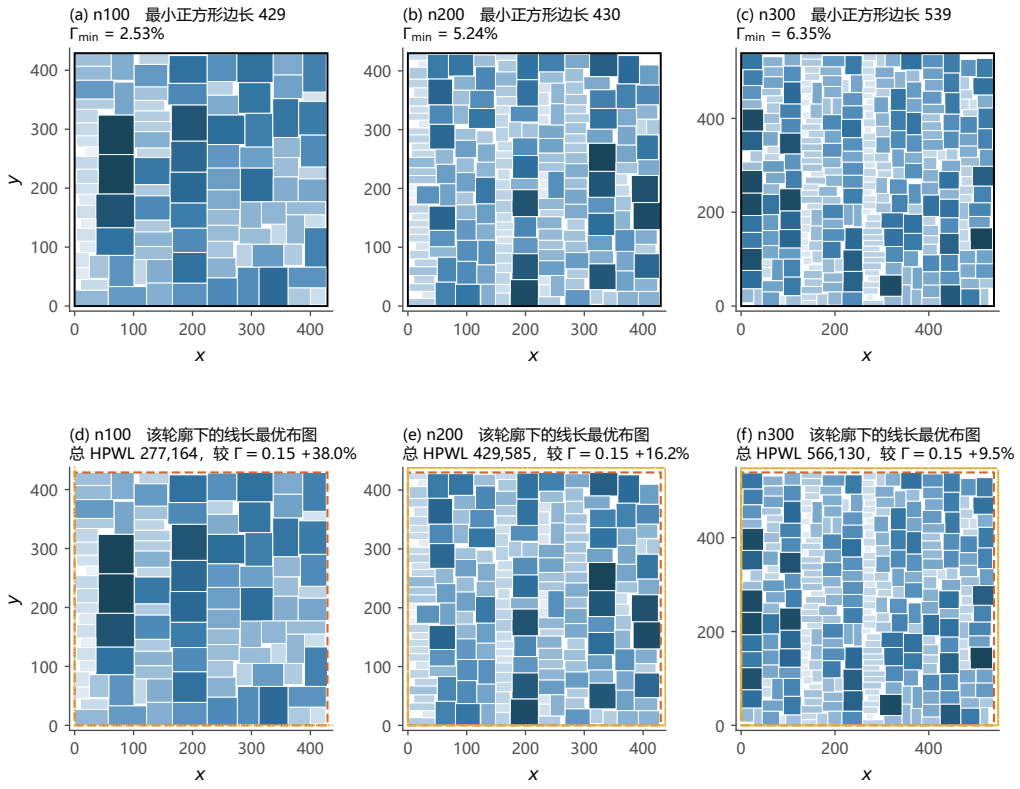


图 25　问题三结果图：最小死区比例对应的布图，以及该轮廓下线长最优的布图

最小正方形面积不小于问题一的包围矩形面积对应的下界，三组数据均满足这一序关系。三问结果的量化对比见图 26：死区比例随模块数上升，n100 与 n200 低于参考文献报告的水平；轮廓收到最紧时线长上升约五成。

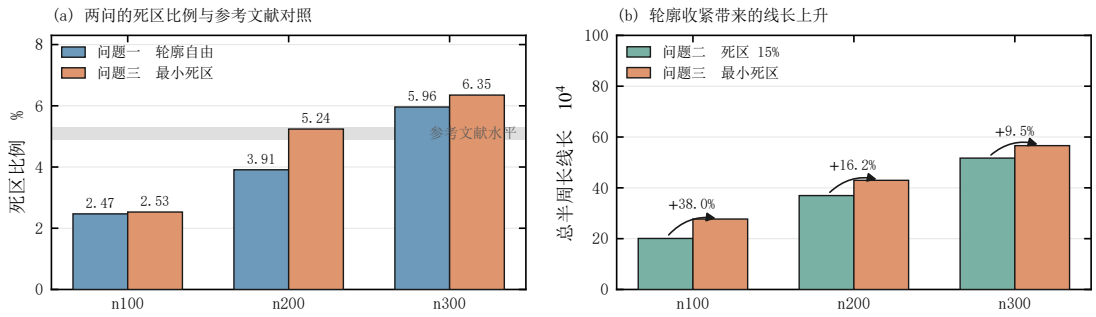


图 26　三组芯片的结果对比。(a) 问题一与问题三的死区比例，灰带为参考文献报告的水平；(b) 轮廓由死区 15% 收到最小值时总线长的上升

6.4　问题四模型的建立与求解

6.4.1　问题四思路框架

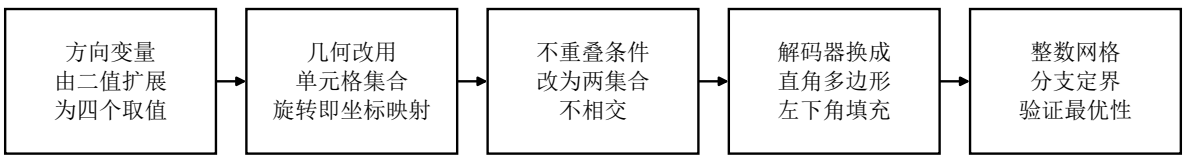


图 27　问题四思路框架图

6.4.2 直角多边形模块模型的建立

模块由矩形变为 L 型与 T 型后，问题一的模型需要修改三处，其余部分保持不变。

修正一：方向变量与几何表示. 方向变量由 $r_i \in \{0,1\}$ 扩展为 $r_i \in \{0,1,2,3\}$，分别对应旋转 0、90、180、270 度。模块 i 的几何用单元格集合 $C_i \subset \mathbb{Z}^2$ 描述，旋转 90 度对应映射 $\rho:(x,y)\mapsto(-y,x)$，旋转 r_i 次后再平移到第一象限，记作

$$C_i(r_i)=\operatorname{norm}\big(\rho^{r_i}(C_i)\big),\qquad \operatorname{norm}(C)=\Big\{\big(x-\min_{C}x,\ y-\min_{C}y\big)\ :\ (x,y)\in C\Big\}. \tag{16}$$

修正二：不重叠约束. 两个直角多边形不重叠等价于平移后的单元格集合不相交：

$$\big(C_i(r_i)+\boldsymbol{p}_i\big)\cap\big(C_j(r_j)+\boldsymbol{p}_j\big)=\varnothing,\qquad \forall\, i<j, \tag{17}$$

其中 $\boldsymbol{p}_i=(x_i,y_i)$。若把每个直角多边形拆成若干矩形的并，上式就退化为矩形之间两两不重叠，与式(4)同形，只是约束条数由 $O(n^2)$ 变为 $O\big((\sum_i m_i)^2\big)$，$m_i$ 为模块 i 的矩形分解块数。

修正三：解码器. B*-tree 的轮廓打包假设模块是矩形，遇到凹形模块时凹口填不进东西，会白白浪费面积。因此把解码器换成直角多边形的左下角填充：按给定顺序逐个放置模块，每个模块取最靠下、同高时最靠左的可行位置。搜索算法沿用问题一的模拟退火，扰动改为交换摆放顺序和改变方向变量。

于是问题四的模型为

$$\begin{cases}\min\quad W\cdot H\\[2mm] \text{s.t.}\ \begin{cases}\big(C_i(r_i)+\boldsymbol{p}_i\big)\cap\big(C_j(r_j)+\boldsymbol{p}_j\big)=\varnothing, & \forall\, i<j\\ W=\max\limits_{i}\ \max\limits_{(x,y)\in C_i(r_i)+\boldsymbol{p}_i}(x+1),\quad H=\max\limits_{i}\ \max\limits_{(x,y)\in C_i(r_i)+\boldsymbol{p}_i}(y+1), & \\ \boldsymbol{p}_i\in\mathbb{Z}^2_{\ge 0},\quad r_i\in\{0,1,2,3\}. & \end{cases}\end{cases} \tag{18}$$

6.4.3 模型的求解

题给四个模块的尺寸经读图并用像素测量复核后列于表 5。全部边长都是整数，把布局限制在整数网格上不损失最优性，因此可以做精确的分支定界。具体做法是按面积从大到小依次放置，对每个模块枚举全部朝向与全部可行位置；一旦某个包围矩形能放下全部模块，即得最优解。

表 5　问题四中四个模块的几何信息

模块	形状	包围盒	面积
b_1	T 型，横条 4×2 在上，竖脚 2×2 居中	4×4	12
b_2	L 型，左列 1×4，右下 1×2	2×4	6
b_3	矩形	2×1	2
b_4	矩形	1×4	4
合计			24

四块总面积 24 是包围矩形面积的下界。按面积从小到大枚举包围矩形，分支定界在面积 24 处找到可行摆放，因此最小包围矩形面积恰为 24，下界可以取到，死区为零。达到该面积的轮廓只有 4×6 与 6×4 两种，长宽比 1.5。一种具体摆法为：b_1 旋转 180 度，横条占据顶部两行的整个宽度，竖脚向下伸入中间；b_2 保持原方向放在左下角；b_3 旋转 90 度变成 1×2 填在中间下方；b_4 保持 1×4 竖放在右侧。

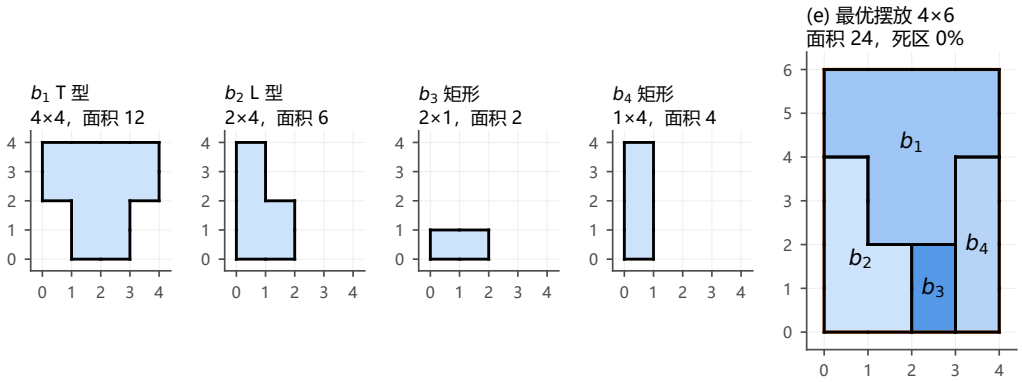


图 28　问题四结果图：四个模块的几何与最优摆放

四个模块的几何与这一摆法见图 28，按单元格看的嵌合关系见图 29。

面积等于下界 24 的候选轮廓共有八个，枚举结果见图 30。其中只有 4×6 与 6×4 能放下四个模块，其余六个都因为某一边太窄而容不下 b_1 的 4×4 包围盒。这也说明最小面积对应的轮廓形状是唯一确定的。

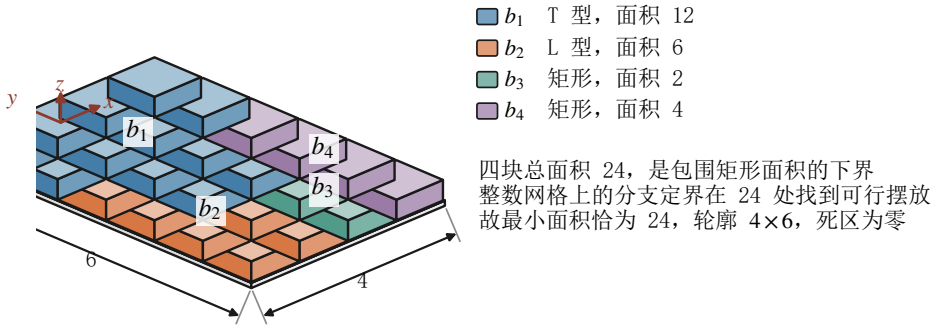


图 29　问题四的最优摆放。四个模块按单元格嵌进 4×6 轮廓，恰好填满，死区为零

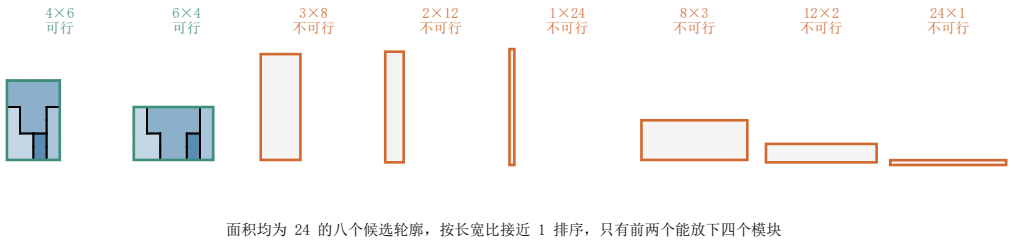


图 30　面积均为 24 的八个候选轮廓的枚举结果

6.4.4　模型的检验

其一，最优性证明。总面积 24 是理论下界，而分支定界在 24 处找到了可行解，因此该解是全局最优，不依赖启发式算法的运气。

其二，两条路线互证。用修正后的模型配合退火与左下角填充解码器求解，得到的最小面积同样是 24、轮廓同样是 4×6，与分支定界一致，说明模型修正是正确的。

其三，几何校验。摆放结果按单元格逐格检查，四个模块的单元格集合两两不相交、并集恰好铺满 4×6 的网格，无重叠也无空洞。

七、算法对照实验

选用模拟退火不是想当然。本节把它和另外五种常见启发式放在同一条件下比较：同一目标函数、同一套 B*-tree 打包与半周长线长计算、同样的目标函数求值次数预算，每种方法各做 16 次独立重启。除切片式退火用波兰表达式外，其余方法都作用在 B*-tree 上，邻域操作也完全相同，因此差别只来自搜索策略本身。

7.1 五种对照方法

遗传算法。个体是一棵 B*-tree。交叉用顺序交叉算子作用在节点承载的模块序列上，即保留父代甲的一段，其余位置按父代乙的相对顺序补齐，保证子代仍是一个排列；树形从父代之一继承，模块朝向逐位随机取自两个父代。变异沿用退火的三种邻域操作，每次施加一到三步。选择用三元锦标赛，每代保留两个精英。

禁忌搜索。每轮在当前解周围采样 24 个邻居，取其中非禁忌的最好者作为下一步。禁忌表记住最近 20 次移动涉及的节点，命中比历史最优更好的解时解除禁忌。

多起点爬山。只接受严格改进的移动，连续 $6n$ 次找不到改进就换一个随机起点重来。它相当于去掉了退火的接受劣解机制，用来度量这一机制值多少。

切片式退火。改用波兰表达式作表示：后缀式序列里操作数是模块，H 表示上下叠放、V 表示左右并排，解码时自底向上合并包围盒。移动沿用 Wong 与 Liu 的三种算子并保持归一化。这种表示只能描述可以用一刀切递归分割的布图，是一个刻意受限的对照组。

构造式贪心。模块按面积降序，逐个放到当前天际线上最低最左的合法位置，两个朝向都试。没有搜索过程，改为跑 300 个不同的模块顺序取最好，作为参照基线。

7.2 对照结果

问题一比死区比例，结果见表 6；问题二比总线长，见表 7。16 次重启的分布见图 31。

表 6　六种启发式在问题一下的死区比例（%），同一求值次数预算，16 次独立重启

方法	n100		n200		n300	
	最好	中位数	最好	中位数	最好	中位数
模拟退火	4.29	5.51	6.43	9.56	7.50	10.34
遗传算法	7.07	8.27	9.93	11.15	11.49	12.77
禁忌搜索	12.31	13.66	14.06	15.78	15.20	16.47
多起点爬山	10.55	11.97	16.28	16.79	17.03	18.37
切片式退火	17.11	20.39	20.70	25.69	22.45	26.83
构造式贪心	11.04	13.06	10.44	10.94	11.34	12.15

表 7　六种启发式在问题二下的总线长，同一求值次数预算，16 次独立重启

方法	n100		n200		n300	
	最好	中位数	最好	中位数	最好	中位数
模拟退火	219 036	268 775	393 500	412 050	849 630	867 539
遗传算法	211 792	232 804	444 150	461 643	676 000	700 810
禁忌搜索	273 178	281 116	502 425	515 452	752 163	752 163
多起点爬山	223 846	233 858	427 184	437 586	611 404	626 839
切片式退火	无可行解		无可行解		无可行解	
构造式贪心	284 362	288 028	541 373	547 752	821 732	831 423

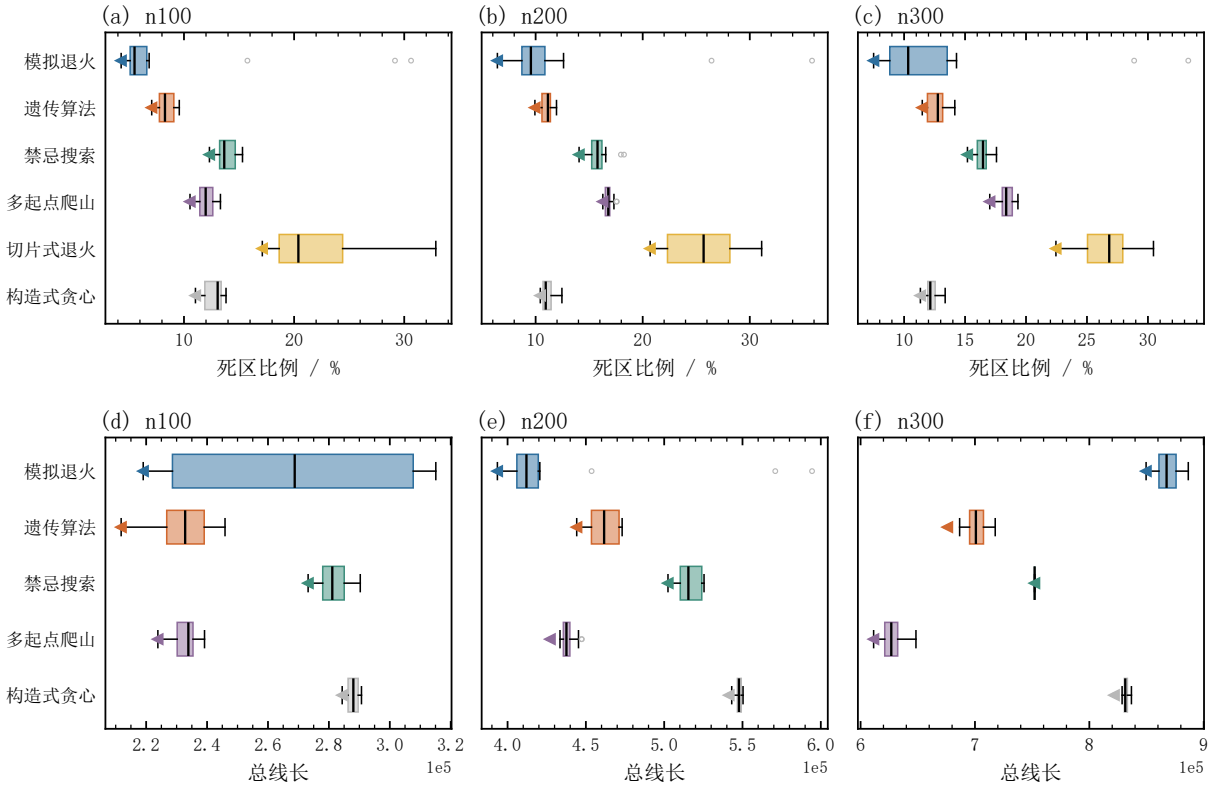


图 31　六种启发式各 16 次独立重启的结果分布，左向三角为最好值

问题一上三组算例的排序一致，模拟退火最好，遗传算法次之，切片式退火最差。把最好的四个布局并排画出来，差别一眼可见，见图 32：切片式的布图呈明显的横向条带，因为波兰表达式只能表达一刀切的分割结构，无法把模块错开咬合；构造式贪心呈货架状，每层高度由该层最高的模块决定，层与层之间留下成片空隙；模拟退火与遗传算法的布图则看不出规则结构，模块互相嵌合。这说明死区的差距主要来自表示能力与嵌合程度，而不是数值上的微调。

多重启能补回多少，见图 33。纵轴是按次序统计量算出的重启 k 次的期望最好值。模拟退火的曲线在前三次重启下降最快，之后趋平；其余方法的曲线几乎从一开始就是平的。这说明退火的单次结果方差大但优质解出现的概率高，适合多重启取优；爬山与贪心的方差小，但小在都不够好，多跑也捞不回来。

7.3　两点值得说明的结果

其一，切片式退火在问题二下 16 次重启全部失败，没有一个解能装进死区比例 0.15 的轮廓。原因不是搜索不力，而是表示本身的限制：切片布图的死区下界明显高于一般

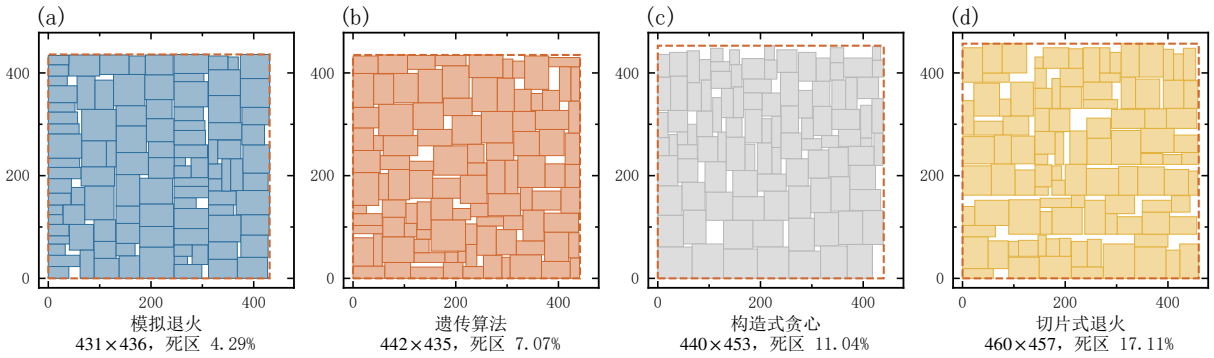


图 32　四种方法在 n100 问题一下的最优布图，橙色虚线为各自的包围轮廓

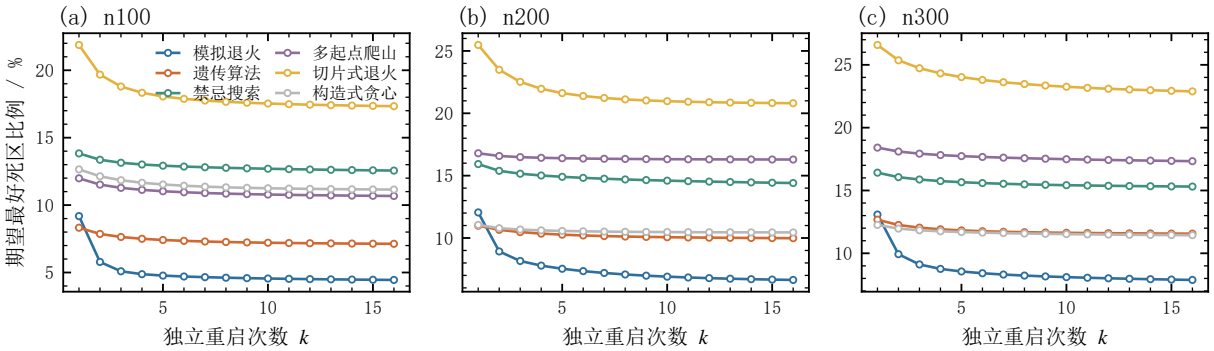


图 33　重启次数与期望最好死区比例的关系

布图，本文三组算例上它连问题一的自由轮廓都要留出 17% 以上的死区，自然进不了只留 15% 的固定轮廓。这从反面说明选择非切片表示是必要的。

其二，两个目标下的胜者不是同一个。把预算从 50 万次求值加到 800 万次再比一遍，结果见图 34。

问题一上模拟退火在三档预算下都最好，死区比例从 5.21% 降到 4.67%，遗传算法改善最快，从 9.31% 降到 5.32%，但始终没有追上。禁忌搜索与构造式贪心的曲线是水平的：贪心本来就没有搜索过程，禁忌搜索则在几万次求值内就锁死在一个局部解上，再给预算也用不掉。

问题二的情况相反。遗传算法从 24.6 万降到 21.0 万，在 800 万次时反超退火；退火的曲线不单调，因为这一档只做 6 次重启，而退火在线长目标下的重启方差很大，取最好值本身就是个高方差的统计量。这说明退火的优势主要在面积类目标上，线长目标下它并没有稳定的优势。因此本文在问题二的最终结果上，对退火、遗传算法与多起点爬山在同一满预算下各跑一遍，取三者中最好的，而不是默认用退火。哪种方法胜出见表 3的备注。

八、模型的灵敏度分析

本文对两个关键参数做了灵敏度分析，另外单独讨论死区比例的影响。

第一个是问题一代价函数中的长宽比权重 λ。固定其余设置，取 $\lambda \in \{0.1, 0.5, 1, 2, 5\}$ 在 n100 上各跑 20 次重启，结果见图 36(a)。$\lambda = 0.1$ 时退火滑向细长条布局，长宽比达到 1.66，死区比例反而升到 8.41%；λ 增到 0.5 以上，长宽比落到 1.1 附近，死区比例稳定在 3.5% 到 4.2% 之间，两项都不再随 λ 明显变化。也就是说结论在一个量级的范围内是稳定的，本文取 $\lambda = 2$ 位于该稳定区间。

第二个是重启次数 k。模拟退火是随机算法，单次运行的结果有方差，重启次数决定了取优能挖到多深。本文在 n100 与 n200 上各做 40 次独立重启并记录每一次的结

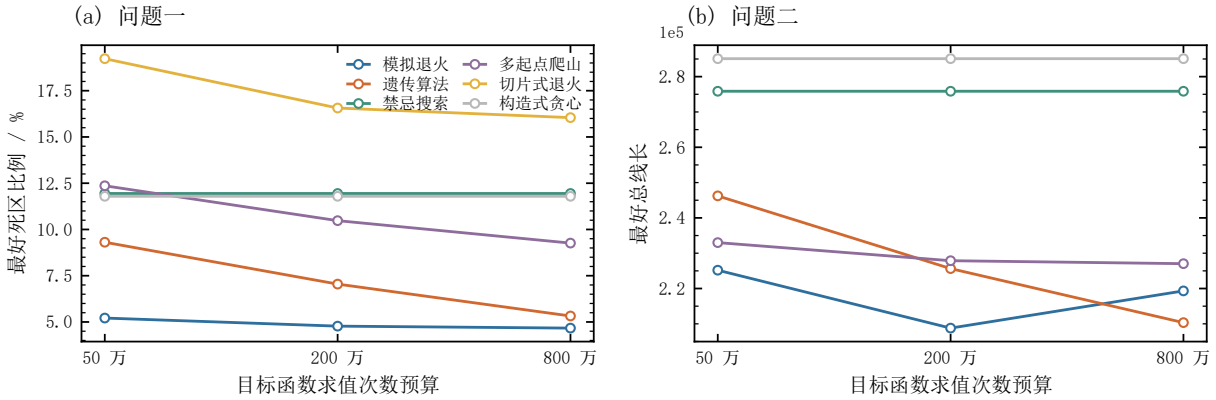


图 34　求值次数预算与最优结果的关系，n100

果，再由这 40 个样本推出取 k 次重启时最好解的期望与十分位带，见图 36(b)。曲线在 k 小于 10 时下降很快，单次重启的期望结果比 40 次重启的最好解差 39% 到 44%；k 到 20 至 25 次之后基本不再改善。这说明本文采用的 20 次重启已经接近该预算下的收益拐点，继续增加重启带来的改善有限，而十分位带的宽度也说明单次运行的结果不可靠，多重启取优是必要的。

第三个是死区比例 Γ 对线长的影响。表 4 只给出两个端点，中间的形状要靠逐点求解才能看清。本文取 $\Gamma \in \{0.04, 0.06, 0.08, 0.10, 0.15, 0.20\}$，把对应的轮廓边长向上取整后作为硬约束，每一档都从同一棵 B*-tree 出发、用同样的预算与重启次数做一次线长最小化，结果见图 35。这样每个点的优化力度一致，曲线才有可比性；直接扫 Γ 而让每档独立随机初始化，求解方差会盖过参数本身的效应，曲线甚至不单调。

三组算例的曲线形状一致：Γ 从 0.20 降到 0.08 时线长上升平缓，再往下收紧则急剧上升。原因是轮廓从宽松收到 0.08 附近时，模块之间仍有足够空隙供线长优化调整位置；继续收紧则空隙被挤干，布局的自由度主要花在装得下上，线长只能被动接受。这说明面积与线长的取舍不是线性的，把死区压到理论下限的最后几个百分点，代价远高于前面一大段。

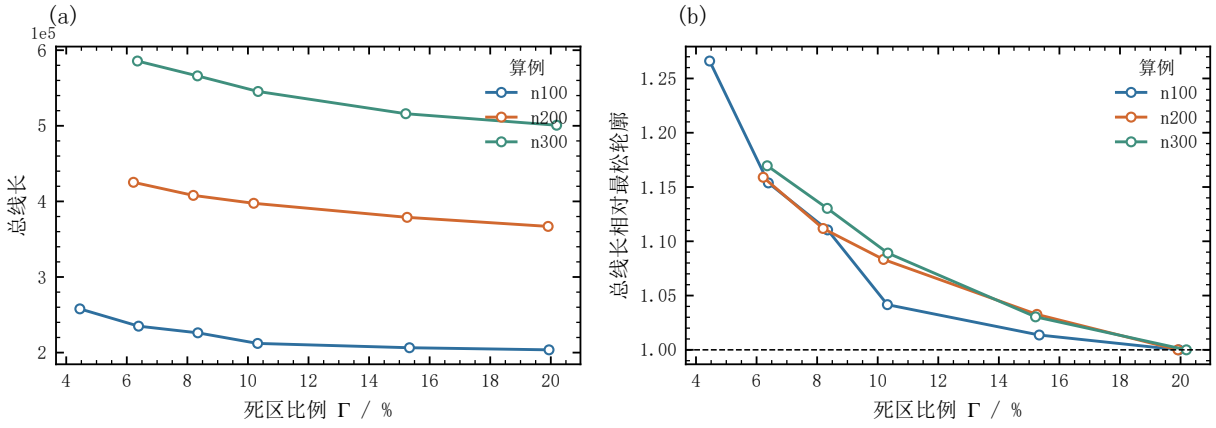


图 35　死区比例与总线长的权衡曲线，每个点都是该轮廓下独立求解的结果

九、模型的评价、改进与推广

9.1　模型的优点

1. 用 B*-tree 表示把析取的不重叠约束从模型中消去，使 300 个模块的算例可以在分钟级求解，而等价的混合整数规划需要约 1.8×10^5 个附加 0-1 变量。

2. 四个小问共用一套表示与一套搜索算法，只更换代价函数或解码器，模型之间衔接

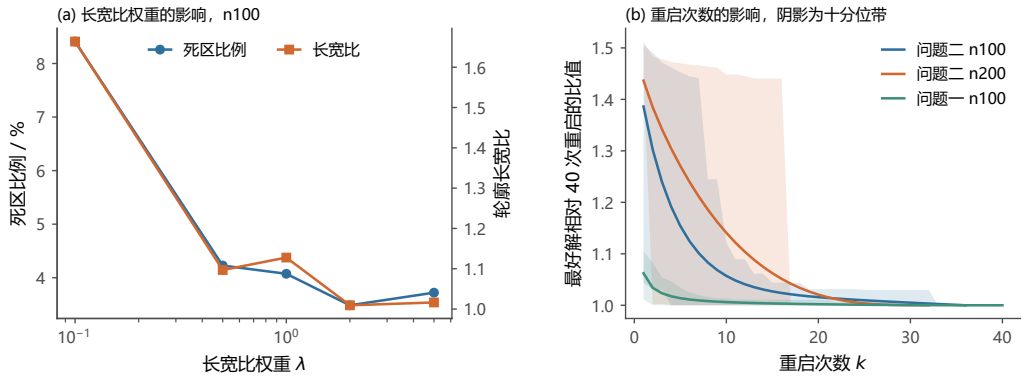


图 36　灵敏度分析：长宽比权重与重启次数的影响

自然，实现与调参的成本低。

3. 问题三通过单调性证明把二维搜索降为一维，省掉了对死区比例的外层二分，计算量减少一个数量级。

4. 结果由一份不复用求解器代码的独立程序校验，几何合法性与线长数值都可复现。

9.2　模型的缺点

1. 模拟退火是启发式算法，只能给出上界，无法证明所得布局是全局最优；问题三报告的最小死区比例同样只是上界。

2. B*-tree 解码出的布局总是紧贴左下角，天然偏向紧凑而非均匀铺开，固定轮廓下需要靠后处理才能利用剩余空隙。

3. 半周长线长忽略实际布线的绕行与过孔，也没有考虑布线拥塞，因此线长最优的布图未必是布线最容易的布图。

9.3　模型的改进

一是把软模块纳入模型。实际设计中部分模块面积固定而长宽比可调，只需在扰动中增加调整长宽比的操作，代价函数不变。二是在退火之外增加一层解的重组，例如把多次重启得到的优质布局按区域拼接后再局部优化，可以缓解单次退火方差偏大的问题。三是把线长模型换成考虑布线资源的加权形式，对拥塞区域的线网加大权重。

9.4　模型的推广

本文的框架适用于任何在给定容器内放置互不重叠的刚性件并优化某个全局指标的问题。把 HPWL 换成物流成本即得设施布局问题，换成热耦合即得三维封装的散热优化；把二维轮廓换成三维体素、把 B*-tree 换成对应的三维表示，即可推广到多层芯片堆叠。问题四给出的直角多边形处理方式，也可直接用于板材下料与不规则件排样。

AI 工具使用声明

十、参考文献

参考文献

[1] 半周长线长 HPWL. https://www.bohrium.com/sciencepedia/feynman/keyword/half_perimeter_ 访问时间 2026 年。

[2] Chen T C, Chang Y W. Modern Floorplanning Based on B*-Tree and Fast Simulated Annealing. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 25(4): 637–650, 2006.

[3] 黄志鹏, 李兴权, 朱文兴. 超大规模集成电路布局的优化模型与算法. 运筹学学报, 25(3): 15–36, 2021.

[4] Chang Y C, Chang Y W, Wu G M, et al. B*-Trees: A New Representation for Non-Slicing Floorplans. Proceedings of the 37th Design Automation Conference: 458–463, 2000.

[5] Kirkpatrick S, Gelatt C D, Vecchi M P. Optimization by Simulated Annealing. Science, 220(4598): 671–680, 1983.

附录

附录 A　运行环境

求解器用 C++17 编写,编译命令为 `g++ -O3 -march=native -std=c++17 -pthread`，在 Ubuntu 20.04、20 核处理器上以 20 线程运行。数据解析、结果校验与绘图用 Python 3 完成，依赖 numpy 与 matplotlib。全部程序与结果文件按下述结构组织：

- `fp.cpp`：B*-tree 表示与模拟退火求解器，支持问题一、二、三三种目标；
- `parse_data.py`：解析.blocks、.nets、.pl 三种文件并做一致性核查；
- `verify_and_plot.py`：独立校验布局并绘图，不复用求解器代码；
- `refine.py`：固定轮廓下的整体平移与滑动精修；
- `reinsert.py`：固定轮廓下的拔出重插精修；
- `q4_solver.py`：问题四的分支定界与退火求解；
- `select_best.py`、`finalize.py`：按题目判优规则选取最终答案并汇总。

复现方式：先运行求解器得到布局文件，再运行 `finalize.py` 完成精修、选优、校验与出图。

附录 B　程序清单

全部源程序随支撑材料一并提交，按问题分目录组织，运行前把各文件开头标注为待填的路径常量与 shell 脚本里的工作目录改成本机路径。

目录	文件	功能
公共模块	`fp.cpp`	B*-tree 表示与模拟退火求解器，支持三种目标，多线程重启
	`parse_data.py`	解析.blocks、.nets、.pl 三种文件并做一致性核查
	`bstree.py`	B*-tree 的 Python 参考实现
	`verify_and_plot.py`	独立校验布局并绘图，不复用求解器代码
	`select_best.py`、`finalize.py`	按判优规则选取最终答案并汇总
Q1	`run_q1.sh`、`collect_results.py`	面积目标的两种权重求解与结果汇总
Q2	`run_q2.sh`、`refine.py`	固定轮廓下的线长优化与滑动精修
Q3	`run_q3.sh`、`collect_sensitivity.py`	最小正方形边长求解与灵敏度汇总
Q4	`q4_solver.py`、`q4_bruteforce.py`	分支定界与退火求解，小规模穷举核对
作图	见画图代码目录	论文全部图件的绘制脚本与统一样式，单独成一个目录随支撑材料提交